Herald

Protocol Research: WebSocket (RFC 6455)

Field	Value
Revision	1
Created	2026-05-22
Last modified	2026-05-22
Status	research-only (no implementation yet)
Status summary	WebSocket (RFC 6455, 2011) is the de-facto standard for bidirectional, full-duplex communication over a single TCP connection. For Herald, WebSocket is the right surface for INTERACTIVE flows: iherald incident chat, sherald live safety state dashboard, browser-based admin panels. Use `github.com/coder/websocket` (formerly `nhooyr.io/websocket`) — actively maintained; `gorilla/websocket` is archived since late 2022. Wave 4e (parallel with gRPC + SSE).
Issues	open-questions: ping/pong cadence; max payload; per-message Brotli compression.
Continuation	Wave 4e — open HRD-291..HRD-302; new module `commons_ws/`.

Constitutional anchors

- **§107 anti-bluff** — WebSocket tests use real `websocat` CLI + browser-based test client; assert frame parsing + ping/pong.
- **§11.4.74 catalogue-check** — performed; no `vasic-digital` or `HelixDevelopment` WebSocket module. Verdict: use `github.com/coder/websocket` via `go get` (well-known modern Go WS library, idiomatic, context-aware).
- **§11.4.61** — tracked doc.

Table of contents

§1. Protocol overview

WebSocket (RFC 6455, 2011). A TCP-based protocol layered on top of HTTP for the initial handshake, then upgrading the connection to a long-lived bidirectional frame-based message channel. Standard in every browser.

Wire format. After the HTTP Upgrade handshake, the connection carries WebSocket FRAMES:

- Header: opcode (text/binary/close/ping/pong), payload length, masking bit + key (client-to-server frames MUST be masked).
- Payload: text or binary, up to 2^63-1 bytes.

Handshake.

```
GET /ws HTTP/1.1
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: <base64-16-bytes>
Sec-WebSocket-Version: 13
```

Server responds with 101 Switching Protocols + `Sec-WebSocket-Accept`.

Authentication. Not built-in. Common patterns: JWT in initial HTTP query (`?token=...`), in a custom HTTP header, or in the first WebSocket message. Cookies work for browser flows.

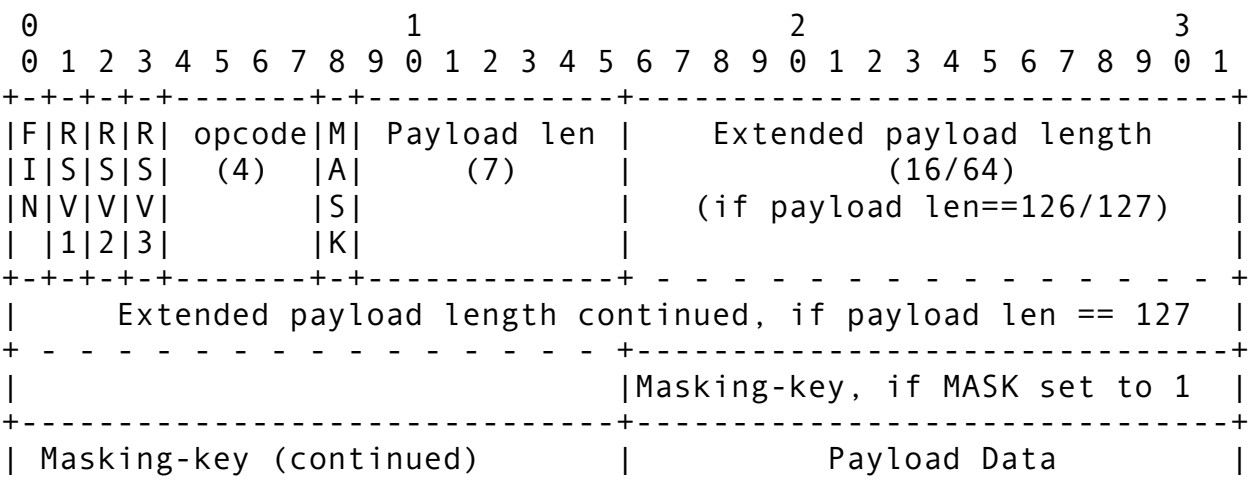
Extensions. Permessage-Deflate (RFC 7692) for compression. Subprotocol negotiation via `Sec-WebSocket-Protocol`.

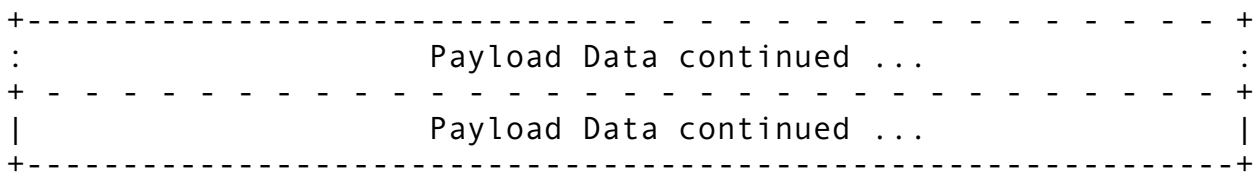
Connection lifecycle. OPEN → CLOSE (graceful) or CLOSE_ABRUPT. Ping/pong keepalive at app-defined cadence.

Adoption. Browser-native. Slack, Discord, OpenAI Realtime, GitHub PRs (live updates), most chat apps.

§2. Specification deep-dive

§2.1 Frame structure





Opcodes:

- 0x0 — continuation
- 0x1 — text (UTF-8)
- 0x2 — binary
- 0x8 — close
- 0x9 — ping
- 0xA — pong

§2.2 Ping/pong

Server SHOULD send ping at a configurable cadence (e.g. every 30s); client MUST reply with pong. Detects dead connections. Not optional — without it, idle TCP can hang for hours.

§2.3 Close handshake

Either party sends a close frame (opcode 0x8) with optional reason code + body. Peer MUST respond with close. Then TCP FIN.

§2.4 Subprotocols

Application-layer protocols (e.g. `chat`, `mqtt`, `wamp`) negotiated via `Sec-WebSocket-Protocol` header.

§3. Herald-specific applicability analysis

§3.1 Where WS fits

- **iherald incident chat** — real-time multi-participant incident war room. Customers + Herald operators + LLM agents exchange messages.
- **sherald live safety dashboard** — browser dashboard subscribes to `/ws/safety` for live updates as safety events land.
- **cherald compliance dashboard** — similar, live posture changes.
- **pherald event stream** — browsers viewing event timelines.

§3.2 SSE vs WebSocket

- **SSE** is one-way (server-to-client), simpler, works over plain HTTP/2, no upgrade dance.
- **WebSocket** is bidirectional, supports text + binary, can carry custom subprotocols.

Use SSE when client doesn't need to talk back; use WS when interactive. Herald uses BOTH (SSE for dashboards; WS for chat).

§3.3 Multi-tenant + auth

JWT in query string (`?token=`) for browser flows (browsers can't set Authorization on WS upgrade) OR JWT in cookie. Validate during handshake; reject with 401 BEFORE upgrade. Bind `tenant_id` from claim onto the connection state.

§3.4 Idempotency

Per-message: client includes idempotency key in JSON payload. Same Redis SETNX path.

§3.5 OTel propagation

Initial HTTP handshake carries `traceparent`. After upgrade, each message embeds a `traceparent` field in JSON. Herald defines this convention in spec V3 §4v.

§3.6 Failure modes

1. **Stale connections.** Ping/pong detects; configure 30s ping; close after 60s no pong.
2. **Slow clients.** Backpressure via bounded channel + drop-oldest on overflow.
3. **Mask validation.** Server MUST drop unmasked client frames per RFC 6455 §5.1 — the `coder/websocket` library handles this.
4. **CSRF on browser WS.** Browsers don't enforce CORS on WS upgrade; validate `Origin` header explicitly.

§4. Step-by-step implementation guide for Herald

§4.1 Add dep

`github.com/coder/websocket` via `go get` (per §9.1 carve-out for well-known modern libs).

§4.2 New module `commons_ws/`

```
commons_ws/
├── go.mod
├── server/
│   ├── upgrader.go              # handshake + auth + origin check
│   ├── conn.go                  # connection state (tenant_id, subscripti
│   ├── hub.go                   # multi-conn fan-out hub
│   ├── codec.go                 # JSON codec + idempotency_key extraction
│   ├── otel.go                  # traceparent injection per message
│   └── server_test.go
├── client/
│   ├── client.go                # outbound WS (for tests; not Herald's pr
│   └── client_test.go
└── e2e_real_websocat_test.go
```

§4.3 Subprotocol naming

`herald.events.v1`, `herald.chat.v1`, `herald.safety.v1`. Each subprotocol has its own message schema (JSON Schema documented in spec V3 §4v).

§4.4 Per-flavor routes

Flavor	WS route	Subprotocol	Purpose
pherald	`/ws/events`	`herald.events.v1`	live event timeline
sherald	`/ws/safety`	`herald.safety.v1`	live safety state
cherald	`/ws/compliance`	`herald.compliance.v1`	live posture
iherald	`/ws/incidents/{id}`	`herald.chat.v1`	incident war room

§4.5 New e2e_bluff_hunt invariants

- **E79** — happy path: `websocat` connects with valid JWT; receives event.
- **E80** — JWT-less: `websocat` connects without token; 401.
- **E81** — ping/pong: 30s ping cycle observed; connection stays alive.
- **E82** — bad origin: connect from disallowed Origin → 403.
- **E83** — close handshake: server initiates close; client gets close frame with reason.
- **E84** — backpressure: slow client; server drops oldest messages without crashing.

§4.6 HRD scaffolding

- **HRD-291** — bootstrap `commons_ws/`.
- **HRD-292** — upgrader + auth.
- **HRD-293** — conn state machine.
- **HRD-294** — hub + fan-out.
- **HRD-295** — codec + idempotency.
- **HRD-296** — OTel.
- **HRD-297** — pherald events WS.
- **HRD-298** — sherald safety WS.
- **HRD-299** — cherald compliance WS.
- **HRD-300** — iherald chat WS.
- **HRD-301** — e2e_bluff_hunt E79–E84 + mutation gates + spec amendments.
- **HRD-302** — close Wave 4e (WS slice).

§5. §107 anti-bluff testing strategy

The bar: `websocat` + a real browser EventSource-like test page successfully open WS connections, exchange messages, observe ping/pong, observe close handshake.

§5.1 Happy-path tests

- **Test 1: websocat connect.** `websocat 'wss://localhost:7104/ws/events?token=<jwt>'` → 101 + greeting message.
- **Test 2: bidirectional.** websocat sends text; Herald echoes; assert receive.
- **Test 3: ping/pong.** Connect; wait 35s; assert connection alive AND server emitted ≥1 ping.

§5.2 Edge cases

- **Test 4: large message.** 1 MiB text message → accepted; 10 MiB → rejected (size limit).
- **Test 5: binary frame.** Send binary; assert handled or rejected per subprotocol.
- **Test 6: fragmentation.** Multiple FIN=0 frames assembled into one message.

§5.3 Failure modes

- **Test 7: JWT-less.** Connect without token → 401 during HTTP handshake (before upgrade).
- **Test 8: bad Origin.** Origin `https://evil.com` → 403.
- **Test 9: cross-tenant.** Tenant A's JWT subscribes to Tenant B's resource → no messages received.

§5.4 Performance

- **Test 10: 1000 concurrent connections.** All receive broadcast; no message loss; CPU < threshold.

§5.5 Mutation gates

- Mutation: skip auth in upgrader → Test 7 FAILS.
- Mutation: skip Origin check → Test 8 FAILS.
- Mutation: skip ping ticker → Test 3 FAILS.

§5.6 Wire-level

- tcpdump :7104 + Wireshark WS dissector → assert frame structure conforms to RFC 6455.

§6. Open questions for operator

1. **Ping cadence?** Recommend 30s.
2. **Max message size?** Recommend 1 MiB.
3. **Permessage-Deflate compression?** Adds ~5x size reduction for JSON; recommend enabled.
4. **Brotli per-message?** Adds CPU; only enable on text frames > 4 KiB.
5. **Allowed Origins?** Per-tenant config — each tenant declares its allowed Origins.
6. **Auth via cookie or query?** Cookie cleaner for browsers; query for `websocat` / programmatic clients. Recommend BOTH.

§7. References

(All fetched 2026-05-22.)

- **RFC 6455.** https://datatracker.ietf.org/doc/html/rfc6455
- **RFC 7692 (Permessage-Deflate).** https://datatracker.ietf.org/doc/html/rfc7692
- **coder/websocket.** https://github.com/coder/websocket (formerly nhooyr.io/websocket; archived gorilla/websocket).
- **gorilla/websocket (archived).** https://github.com/gorilla/websocket
- **websocket.org Go guide.** https://websocket.org/guides/languages/go/
- **websocat (CLI tool).** https://github.com/vi/websocat

§8. Catalogue-check verdict

Per §11.4.74:

- **vasic-digital:** no module.

- **HelixDevelopment:** no module.

Verdict: no-match. Depend on `github.com/coder/websocket` **via** `go get` **(per §9.1).** No submodule.